

变压器振动三阶段诊断代码思路指南

—— V1（单分支）与 V2（双分支）的统一理解

说明：本指南更偏向“代码结构 + 数据流”的视角，帮助你从头到尾理清 `transformer_three_stage.py` 与 `transformer_three_stage_v2.py` 的实现逻辑。

1. 整体框架：三阶段在做什么？

目标：从现场 8192 点交流变压器振动原始波形出发，构建一个“三阶段”诊断系统：

- (1) 阶段一：完全无监督，从大量 train + val 数据中学习“正常模式”的隐空间表示，得到一个可以给每条样本打“异常得分”的模型（DeepSVDD + VAE）。
- (2) 阶段二：仍然无监督，对所有样本计算异常得分，根据分位数（例如 5% / 99%）得到“高置信正常 / 高置信异常 / 不确定”三个集合，同时可视化得分分布。
- (3) 阶段三：在不“污染”test 标签的前提下，用少量带标签的 VAL 集做参考，决定阈值或训练轻量分类头，然后在 TEST 上只做前向预测，最后与标签对比。

两套实现：

- V1：单分支 Hetero 图像管线——只用 CWT + STFT + 时域 Context 形成 $3 \times 224 \times 224$ 图，ResNet18 编码后接 DeepSVDD 与空间 VAE。
- V2：双分支管线——Hetero 图像分支 + Zerone 1200 维特征分支，使用 DualBranchEncoder 融合成 512 维表示，再接 DeepSVDD + VAE 头。

理解代码时，围绕四个核心对象即可：

- ThreeStageConfig / ThreeStageConfigV2：全局配置与路径。
- DualBranchDataset：从 JSON/JSONL 振动记录到“图像 + 特征条”的映射。
- HybridAnomalyModel / HybridAnomalyModelV2：编码器 + SVDD 头 + VAE 头。
- train_stage1 / run_stage2 / train_stage3：三阶段训练与推理的调度函数。

2. 数据流：DualBranchDataset 与特征提取（V2 为主）

2.1 输入文件与基础解析

假定每个 JSON/JSONL 记录大致形如：

```
{
  "data_time": "...",    # 时间戳，字段名可能是 dataTime / timestamp / time 等
  "signal_value": "0.12,0.10,...", # 或者是浮点数组
  "sensor_id": "..."
}
```

同一个文件包含多条时间戳记录；同一时间戳下可能有多个传感器（多通道）。

DualBranchDataset(root_dir, cfg, use_labels=False/True) 的索引构建逻辑：

- 1) 遍历 root_dir 下所有 *.jsonl / *.json 文件。
- 2) 读取文件，按照时间字段分组，每个 time_key 形成一个“时间片”。
- 3) 对每个时间片：
 - 把所有记录的 signal_value 解析为 float32 数组，并统一到 8192 长度（截断 / 补零）。
 - 得到若干通道：[sig_ch0, sig_ch1, ..., sig_chU-1]。
- 4) 生成一条样本：(文件路径, 时间 key, 通道列表, label 或 None), 加入 self.samples。

如果 use_labels=True，则通过路径中的关键词（normal / fault / 正常 / 故障 等）推断标签；否则标签统一记为 -1，仅在后续 Stage1/2 做无监督建模时使用。

2.2 从多通道波形到“双分支输入”

在 __getitem__ 中，DualBranchDataset 对每个时间片执行三步：

- (1) 多通道能量加权：
 - 把所有通道堆叠为矩阵 X，形状 (T, U)。T=8192，U 为通道数。
 - 计算每个通道能量 $E_u = \text{mean}(\text{sig}_u^2)$ ，归一化得到权重 $w(u)$ 。
 - 做加权和： $x = X @ w$ ，得到一个“虚拟单通道”信号 x，长度仍为 8192。
- (2) Hetero 图像分支（图像输入）：
 - 对 x 做 z-score 标准化。
 - 构造三条通道：
 - Ch0: CWT（Morlet 小波）时频图；
 - Ch1: STFT 幅度谱图；
 - Ch2: 把 8192 点按行折叠，形成“时域 Context 图像”。
 - 每个通道 resize 到 224×224 ，并做 min-max 归一化。
 - 堆叠成 (3, 224, 224) 的浮点张量。
- (3) Zerone 特征分支（手工特征条）：
 - 调用 zerone_features.py 计算 1200 维特征：
 - 时域特征：约 15 维；
 - STFT 段统计：约 127 维；
 - Welch PSD 压缩特征：约 1050 维；
 - 高频指标 HF 向量：约 8 维。
 - 可以视情况对整条向量做标准化 / 归一化。

最终 __getitem__ 返回：

3. 模型主体：双分支编码器与 HybridAnomalyModelV2

3.1 双分支编码器结构

V2 的编码器可理解为三段：图像分支 + Zerone 分支 + 融合层。

- 图像分支 (CNN)
 - 使用 ResNet18 主干：conv + bn + relu + maxpool + layer1..4。
 - 尾部接 AdaptiveAvgPool2d(1) + Flatten, 输出 512 维特征向量。
- Zerone 分支 (MLP)
 - Linear(1200 -> 512) + BN + ReLU
 - Linear(512 -> 256) + BN + ReLU
 - 得到 256 维特征。
- 融合层
 - 将 512 维 (图像) 与 256 维 (Zerone) 拼接为 768 维。
 - 通过 Linear(768 -> 512) + BN + ReLU 压回 512 维。
 - 输出 512 维统一隐空间表示 z_{enc} , 供 SVDD 与 VAE 共用。

3.2 SVDD 头 (DeepSVDD)

DeepSVDD 在 512 维空间中维护一个中心向量 c :

$$\text{score_svdd}(x) = \|z_{enc}(x) - c\|^2$$

训练时, SVDD loss 通常为 score_svdd 的 batch 均值 (或 soft-boundary 形式), 推动 “多数样本” 靠近中心, 少数 “异常样本” 远离中心。

3.3 VAE 头 (空间结构 + 可解释重构)

为增强结构性与可视化, 模型加入 VAE 解码分支:

- 编码端得到 μ 与 $\log\sigma$ (可对空间特征图做 1×1 卷积)。
- 重参数化: $z = \mu + \epsilon \sigma$ 。
- 解码端使用多层 ConvTranspose2d 逐步上采样回 $3 \times 224 \times 224$ 图。

VAE 损失:

$\text{recon_loss} = \text{L1}(\text{重构图}, \text{输入图})$ 的像素平均;
 $\text{kl_loss} = \text{KL}(N(\mu, \sigma) \| N(0, I))$ 的空间 + 通道平均;
 $\text{loss_vae} = \text{recon_loss} + \beta * \text{kl_loss}$ 。

3.4 综合异常得分

HybridAnomalyModelV2.anomaly_score(image, zerone) 一般组合两部分:

score_svdd = DeepSVDD 的距离得分;
 $\text{score_vae} = \text{recon_loss} + \gamma * \text{kl_loss}$ 。

归一化后, 最终异常得分写成:

$$\text{score_final} = \alpha * \text{score_svdd} + (1 - \alpha) * \text{score_vae}$$

其中 α 由配置 ThreeStageConfigV2 控制。后续 Stage2/Stage3 都基于这个得分。

4. 阶段一 train_stage1_v2：无监督训练流程

4.1 数据使用方式

阶段一把 TRAIN 与 VAL 全部当成“无标签数据”：

```
train_ds = DualBranchDataset(TRAIN_DIR, cfg, use_labels=False)
val_ds = DualBranchDataset(VAL_DIR, cfg, use_labels=False)
combined = ConcatDataset([train_ds, val_ds])
```

这一点很重要：即使 VAL 文件夹里真实上有标签信息，Stage1 也完全不读取，保证“表示学习”和“异常得分定义”是纯无监督的。

4.2 VAE 预训练

为了让重构网络先收敛到“像模像样”的水平，代码通常会先单独训练 VAE 若干 epoch：

- 优化器只更新编码器 + VAE 解码器参数。
- 每个 batch：

```
img, zerone, _, _ = batch
out = model(img, zerone)
loss = recon_loss + 0.01 * kl_loss
```

这样做的直觉是：先把“重构任务”做好，再引入 SVDD 约束，可以避免一开始就把特征空间“拉成一团”的塌陷。

4.3 SVDD 中心初始化

预训练后，需要初始化 SVDD 的中心 c：

- 遍历整个 combined 数据集，前向一次，只取 z_enc。
- 对所有 z_enc 求均值，得到 center 向量。
- 对接近 0 的维度做一个很小的偏移，避免中心刚好在原点。

4.4 联合训练 (SVDD + VAE)

接下来进入完整联合训练阶段：

对于 epoch $e = 1..E$ ：

```
beta = min(BETA_VAE, BETA_VAE * (e / BETA_WARMUP))
```

对每个 batch：

```
out = model(img, zerone)
svdd_loss = mean(score_svdd)
vae_loss = mean(recon_loss) + beta * mean(kl_loss)
total_loss = svdd_loss + vae_loss
```

反向传播 total_loss，做梯度裁剪，更新参数，并通过 CosineAnnealingLR 或其他 scheduler 调学习率。

每轮记录 svdd_loss / vae_loss / total_loss 到 history，并按 avg_total 选出最优模型，保存为 stage1_best.pth。

4.5 阶段一的输出

5. 阶段二 run_stage2_v2：基于异常得分的伪标签

5.1 数据与得分计算

阶段二冻结 Stage1 的模型，再次构造 TRAIN + VAL（仍 use_labels=False）：

```
loader = DataLoader(ConcatDataset([train_ds, val_ds]), batch_size=..., shuffle=False)
```

对每个 batch：

```
img, zerone, _, idx = batch
score = model.anomaly_score(img, zerone)
记录 score 与全局索引 idx。
```

最终得到：

```
all_scores : 每条样本的异常得分
all_indices : 对应的样本索引
```

5.2 分位数阈值与伪标签池

对 all_scores 计算两个分位数阈值：

```
t_normal = percentile(all_scores, NORMAL_PERCENTILE) # 如 5%
t_anomaly = percentile(all_scores, ANOMALY_PERCENTILE) # 如 99%
```

然后划分三类样本索引：

```
pseudo_normal = { idx | score <= t_normal }
pseudo_anomaly = { idx | score >= t_anomaly }
uncertain     = 其余样本。
```

这三类集合构成“伪标签池”：

- pseudo_normal $\approx$ 高置信正常
- pseudo_anomaly $\approx$ 高置信异常
- uncertain $\approx$ 模型尚不自信的样本

5.3 阶段二的输出

Stage2 写出一个压缩文件（如 .npz），包含：

- all_scores, all_indices
- pseudo_normal, pseudo_anomaly, uncertain
- t_normal, t_anomaly

同时绘制：

- 异常得分直方图 + t_normal / t_anomaly 两条竖线；
- 伪标签三类样本占比饼图。

在当前 V2 代码里，Stage3 还没有充分利用这些伪标签，但从设计视角来看，它们是后续“半监督分类头”的天然数据来源。

6. 阶段三：V1/V2 的做法与“标签泄露”风险

6.1 现有 V1/V2 的阶段三逻辑

当前 train_stage3 / train_stage3_v2 的典型流程是：

- 1) 用 use_labels=True 从 TEST 目录构造带标签数据集（通过路径关键词推断）。
- 2) 把 TEST 集拆成 80% 训练子集 + 20% 验证子集。
- 3) 基于 Stage1 的 encoder 构造 FaultClassifier，接一个小分类头。
- 4) 在“TEST 的 80% 子集”上用交叉熵损失训练分类头，
在“TEST 的 20% 子集”上监控 val_acc / val_f1。
- 5) 最后再在完整 TEST 集上评估一次并画混淆矩阵、ROC/PR。

如果原本设计里 TEST 是“最终盲测集”，那这套写法相当于把 TEST 用作了“有标签训练 + 验证”的数据，严格意义上属于“标签泄露”。

6.2 你现在想要的“标签使用规则”

你希望的是：

- TRAIN 与 TEST：训练和打分阶段都当成无标签集，不读取真标签；
- VAL：少量带标签数据，允许在阶段三中参与“阈值选择 / 监督微调 / 指标评估”。

对应 Stage3 有两个自然版本：

版本 A：纯无监督评估

- 仅用 Stage2 的异常得分与阈值，在 VAL 上扫阈选点。
- 固定阈值后，在 TEST 上只做前向预测，输出正常/故障/不确定。
- 如需看 TEST 效果，可在单独评估脚本中用 TEST 标签算一次指标，但不再反向。

版本 B：半监督分类头（可选）

- VAL 标签作为唯一“可信”监督信号。
- 可从 pseudo_normal / pseudo_anomaly 中挑选一批高置信样本，配合 VAL 做训练。
- 分类头训练与 early stopping 只能用 VAL（或其子集），不能用 TEST 标签。
- TEST 仅在训练结束后做一次纯前向评估。

阅读 / 修改 Stage3 代码时，关键就是沿着这条线检查：

“在哪里用到了 use_labels=True？这些标签是用来评估，还是反向更新了参数？”

7. 实战阅读 Checklist：打开三阶段脚本时可以先看什么？

当你再次打开 `transformer_three_stage.py` (V1) 或 `transformer_three_stage_v2.py` (V2) 时，可以按下面这个顺序快速过一遍：

- [1] 配置 `ThreeStageConfig` / `ThreeStageConfigV2`
 - 路径：`PROJECT_ROOT` / `TRAIN_DIR` / `VAL_DIR` / `TEST_DIR`。
 - 训练参数：`BATCH_SIZE` / `LR` / `EPOCHS` / `WEIGHT_DECAY` 等。
 - 异常得分融合参数：`alpha` / `BETA_VAE` / `BETA_WARMUP` 等。
 - 分位数阈值：`NORMAL_PERCENTILE` / `ANOMALY_PERCENTILE`。
- [2] 数据集 `DualBranchDataset` (V2)
 - 确认“样本”的定义：按时间戳聚合通道，还是按文件切。
 - 看清 multi-channel 聚合、pad / truncate、特征提取 pipeline。
 - 检查 Stage1/2 调用时 `use_labels` 是否为 `False`。
- [3] 编码器与头
 - V1：单分支图像编码器 + `DeepSVDD` + `SpatialResNetVAE`。
 - V2：双分支编码器 + `DeepSVDD` + `VAE`，关注 512 维融合向量。
 - 理解 `anomaly_score` 的组合方式。
- [4] Stage1 训练
 - 看 VAE 预训练是否启用。
 - 看 SVDD 中心是如何初始化的。
 - 看损失记录与模型保存逻辑。
- [5] Stage2 伪标签
 - 确认只用了 `TRAIN+VAL`（无标签）。
 - 理解分位数阈值如何选取，输出文件中都保存了什么。
- [6] Stage3 逻辑
 - 查找所有 `use_labels=True` 的地方。
 - 判断当前版本 Stage3 是“纯评估”还是“在 TEST 上监督微调”。
 - 如果要避免标签泄露：确保 TEST 标签只在离线评估脚本中被读取一次。

对照这份 Checklist，你就可以一边看代码一边在脑子里画出“数据流 + 标签流”的双通道图，任何“test 标签参与训练 / 调参”的位置都会非常显眼，方便你按需要重构。